

Student Academic Resource Tracker

Project Documentation and Setup Guide

C# / ASP.NET Core 8 Minimal API · JSON file storage · HTML / CSS / JavaScript · xUnit · Docker

1. Overview

The Student Academic Resource Tracker is a web application where a student records their courses and attaches study resources to them: lecture notes, textbooks, videos, articles, assignments, past questions and links. Each resource has a status (Not started, In progress, Completed), a priority (Low, Medium, High), an optional due date, an optional link, tags and notes.

A dashboard shows the overall completion rate, overdue and due-this-week counts, progress per course, resources by type, open items by priority, and lists of overdue and upcoming deadlines. All resources can be exported to a CSV file that opens cleanly in Excel.

The project is a **monolith**: one ASP.NET Core application serves both the REST API and the front end.

2. Architecture

```
Browser (wwwroot: index.html + app.js + styles.css)
  | JSON over HTTP (/api/...)
ASP.NET Core 8 Minimal API (Program.cs)
  | DataStore.Read / DataStore.Write
JSON data file (Data/tracker.json)
```

Layer	Location	Role
Frontend	<code>wwwroot/</code>	Single-page UI with Overview, Resources and Courses pages, light and dark themes. Plain JavaScript calling the API with <code>fetch()</code> . Served by the same app.
Backend	<code>Program.cs</code>	Startup, REST endpoints, mapping to response objects, CSV export.
Persistence	<code>Services/DataStore.cs</code>	Thread-safe, all-or-nothing reads and writes of the JSON data file.
Reporting	<code>Services/AnalyticsService.cs</code>	Calculates the dashboard figures.
Validation	<code>Services/Validators.cs</code>	Checks all input and cleans up tags.
Error handling	<code>Services/BadRequestExceptionHandler.cs</code>	Turns malformed requests into readable 400 responses.
Models	<code>Models/Entities.cs</code> , <code>Models/Dtos.cs</code>	Stored entities, request bodies and response objects.
Sample data	<code>Services/SeedData.cs</code>	Three courses and eight resources created on first run.

3. Project structure

```

StudentResourceTracker/
├─ Program.cs                App startup + all REST endpoints
├─ StudentResourceTracker.csproj
├─ Models/
│   ├─ Entities.cs          Course, StudyResource, enums, DataFile
│   └─ Dtos.cs              Request and response records
├─ Services/
│   ├─ DataStore.cs         JSON persistence (locking, atomic saves,
rollback)
│   ├─ AnalyticsService.cs   Dashboard calculations
│   ├─ Validators.cs        Input validation
│   ├─ BadRequestExceptionHandler.cs  Malformed request -> 400
│   └─ SeedData.cs          First-run sample data
├─ wwwroot/
│   ├─ index.html           Layout, dialogs and icon set
│   ├─ app.js               Routing, rendering and API calls
│   └─ styles.css           Design system (light and dark themes)
├─ tests/StudentResourceTracker.Tests/  xUnit unit and API tests
├─ docs/DOCUMENTATION.md    This documentation
├─ Dockerfile               Container build used for deployment
├─ render.yaml              Render deployment blueprint
└─ README.md                Quick start

```

4. Prerequisites

- .NET 8 SDK
- A modern web browser
- Optional: VS Code with the C# Dev Kit, Visual Studio 2022 or JetBrains Rider
- Optional: Docker, to build and run the container image locally

5. Setup and run, step by step

1. Install the .NET 8 SDK and confirm with `dotnet --version` (it should print 8.x).
2. Clone the repository: `git clone https://github.com/FelixAshong/StudentResourceTracker-c-.git`
3. Open a terminal in the project folder (the one containing `StudentResourceTracker.csproj`).
4. Run the app: `dotnet run` (packages are restored automatically; internet is needed the first time).
5. Open the URL printed in the terminal, for example `http://localhost:5000`.
6. Stop the app with `Ctrl + C`.

Useful options:

Command / setting	Effect
<code>dotnet run --urls http://localhost:5080</code>	Use a fixed port
<code>dotnet run -- DataFile=/path/to/tracker.json</code>	Store data somewhere else (also settable as the <code>DataFile</code> environment variable)
<code>PORT=8080</code> environment variable	Listen on all interfaces on that port (used by hosting platforms)
Delete <code>Data/tracker.json</code> while stopped	Reset all data; sample data is recreated on the next run

6. Data design

Data is stored as one JSON document, `Data/tracker.json`, with a list of courses and a list of resources. Identifiers are GUIDs generated by the server.

Course

Field	Type	Rules
Id	GUID	Generated by the server
Code	text	Required, max 20 characters, unique (case-insensitive), e.g. CS 301
Title	text	Required, max 120 characters
Lecturer	text	Optional, max 80 characters
Semester	text	Optional, max 40 characters
CreatedAt	date-time (UTC)	Set by the server

StudyResource

Field	Type	Rules
Id	GUID	Generated by the server
CourseId	GUID	Must refer to an existing course
Title	text	Required, max 200 characters
Type	enum	LectureNotes, Textbook, Video, Article, Assignment, PastQuestion, Link, Other
Url	text	Optional; must be an http:// or https:// address
Notes	text	Optional, max 2000 characters
Status	enum	NotStarted, InProgress, Completed
Priority	enum	Low, Medium, High
DueDate	date	Optional, YYYY-MM-DD
Tags	list of text	Up to 10; each trimmed, lower-cased, max 30 characters, duplicates removed
CreatedAt	date-time (UTC)	Set by the server
CompletedAt	date-time (UTC)	Set when the status becomes Completed, cleared if it is reopened

Relationship: one course has many resources. Deleting a course deletes its resources.

Overdue means the resource has a due date before today and its status is not Completed.

How saving works

- Every read and write goes through `DataStore`, which holds a lock so concurrent requests can't interfere.
- A save writes to a temporary file and then swaps it in, so a crash mid-save can't corrupt the data file.

- Writes are all-or-nothing: if a request fails part-way, or the file can't be saved, the in-memory data is restored to the last saved version.
- Requests that change nothing (for example deleting an id that doesn't exist) don't rewrite the file.
- If the data file can't be read at startup, it is renamed to `tracker.corrupt-<timestamp>.json` and the app starts with an empty tracker, so nothing is lost.

7. API reference

All request and response bodies are JSON. Enum values are sent and returned as names (for example `"InProgress"`); numbers are rejected.

Method	Endpoint	Description
GET	<code>/api/courses</code>	List courses, with a resource count for each
POST	<code>/api/courses</code>	Create a course <code>{code, title, lecturer?, semester?}</code>
PUT	<code>/api/courses/{id}</code>	Update a course
DELETE	<code>/api/courses/{id}</code>	Delete a course and its resources
GET	<code>/api/resources?</code> <code>courseId=&type=&status=&priority=&q=&overdue=</code>	List, filter and search resources (q searches title, notes and tags)
POST	<code>/api/resources</code>	Create <code>{courseId, title, type, status, priority, url?, notes?, dueDate?, tags?}</code>
PUT	<code>/api/resources/{id}</code>	Update a resource
PATCH	<code>/api/resources/{id}/status</code>	Change only the status <code>{status}</code>
DELETE	<code>/api/resources/{id}</code>	Delete a resource
GET	<code>/api/analytics/summary</code>	Dashboard figures
GET	<code>/api/export/csv</code>	Download all resources as CSV
GET	<code>/healthz</code>	Health check for hosting platforms

Resources are returned open items first, then by due date (undated last), then by title. Each resource response also includes `courseCode`, `courseTitle` and `isOverdue`.

Responses and errors

Status	When	Body
200 / 201	Success (201 for creates, with a <code>Location</code> header)	The course or resource
204	Successful delete	Empty
400	Validation failed	<code>{"title": "One or more validation errors occurred.", "errors": {"field": ["message"]}}</code>
400	Malformed JSON or unknown enum value	<code>{"title": "The request could not be read.", "detail": "..."}"</code>
404	Unknown id	Problem details
409	Duplicate course code	<code>{"error": "A course with code 'CS 301' already exists."}</code>

Example:

```
curl -X POST http://localhost:5000/api/resources \
-H "Content-Type: application/json" \
-d '{"courseId":"<course id>","title":"Clean Code","type":"Textbook",
    "status":"NotStarted","priority":"High","dueDate":"2026-11-15","tags":
    ["reading"]}'
```

8. Using the application

The app has three pages, reached from the sidebar (or the bottom bar on phones):

1. **Courses:** each course is a card showing its code, lecturer, semester and progress. Add one with *New course* (code and title are required). Hover a card to edit or delete it; deleting asks for confirmation and says how many resources will also be deleted. *View resources* opens that course's list.
2. **Resources:** add a resource with *New resource*: pick a course, enter a title and type, and optionally a status, priority, due date, link, tags (comma separated) and notes.
3. Change progress with the status pill on each row, without opening the edit form. Hover a row to edit or delete it.
4. Filter with the status tabs (which show counts), the course, type and priority menus, *Overdue only*, or the search box (titles, notes and tags). *Clear filters* resets everything.
5. *Export* (or *Export CSV* in the sidebar) downloads every resource as a spreadsheet.
6. **Overview:** a completion ring, counts for in progress, not started, due this week and overdue, a deadline timeline grouped into overdue, this week and later (click an item to edit it), progress per course, the library mix by type, and open items by priority.
7. The sidebar lists your courses; click one to jump to its resources. Toggle dark mode at the bottom of the sidebar (your choice is remembered).

Keyboard shortcuts: `/` focuses search, `N` opens a new resource, `Esc` closes a dialog.

9. Testing

The `tests/StudentResourceTracker.Tests` project contains 31 xUnit tests:

- **Validators:** required fields, length limits, URL schemes, undefined enum values, tag clean-up.
- **DataStore:** seeding, persistence, rollback after a failed write, skipping unchanged saves, corrupt-file recovery.
- **AnalyticsService:** the 7-day window, overdue items not crowding out upcoming ones, completion rates.
- **API:** every endpoint, run in-process with `WebApplicationFactory` against a temporary data file.

Run them with:

```
dotnet test tests/StudentResourceTracker.Tests
```

10. Deployment

The app is deployed to [Render](#) as a Docker web service.

- `Dockerfile` builds the app with the .NET 8 SDK image and runs it on the smaller ASP.NET runtime image as a non-root user.
- `render.yaml` describes the service (free plan, health check on `/healthz`, redeploy on every push to `main`).
- Render sets the `PORT` environment variable, which the app listens on.

To deploy your own copy: in the Render dashboard choose **New** → **Blueprint**, connect the GitHub repository and apply. Render builds the image and gives the service a public `https://...onrender.com` URL.

Limitations of the free plan:

- The file system is not persistent, so the data resets to the sample data whenever the service restarts or is redeployed.
- The service sleeps after about 15 minutes without traffic; the first request after that takes up to a minute.

For permanent data, attach a persistent disk (paid plans) and point `DataFile` at it, or replace `DataStore` with a database (see section 12).

11. Security and design notes

- All user text is HTML-escaped in the front end before it is put into the page, preventing XSS.
- Links are restricted to `http/https`, so a stored link can't run script.
- CSV export prefixes cells starting with `=`, `+`, `-`, `@`, tab or carriage return with `'`, preventing spreadsheet formula injection.
- All input is validated on the server, not just in the browser.

- There is no login. The app is intended for a single student. For multi-user use, add authentication (for example ASP.NET Core Identity) and a user id on every course and resource.

12. Extending the project

- Move to a database: replace `DataStore` with an EF Core + SQLite (or PostgreSQL) implementation. The endpoints only talk to `DataStore.Read` / `DataStore.Write`, so the change is contained.
- Add authentication and per-user data.
- Add file uploads for PDFs and notes.
- Add reminders for upcoming deadlines.

13. Troubleshooting

Problem	Fix
<code>dotnet</code> is not recognised	Install the .NET 8 SDK and reopen the terminal.
Package restore fails	Check your internet connection or proxy settings.
Port already in use	Run <code>dotnet run --urls http://localhost:5050</code> .
Data looks wrong or you want to start over	Stop the app and delete <code>Data/tracker.json</code> .
A <code>tracker.corrupt-...json</code> file appeared	The data file was damaged; the original content is in that file.
Blank page	Run from the project folder so <code>wwwroot</code> is found.
Deployed site takes a long time to load	The free Render service was asleep; wait for the first request to finish.